Fairness-Aware Task Scheduling and Resource Allocation in UAV-Enabled Mobile Edge Computing Networks

Mingxiong Zhao, *Member, IEEE*, Wentao Li, Lingyan Bao, Jia Luo, Zhenli He, and Di Liu, *Member, IEEE*

***Abstract*—Unmanned aerial vehicle (UAV)-enabled mobile edge computing (MEC) has recently emerged to provide data processing and caching in the infrastructure-less areas. However, the limited battery capacity of UAV constrains its endurance time, and makes energy efficiency one of the top priorities in implementing UAV-enabled MEC architecture. In this backdrop, we aim to minimize the UAV's energy consumption by jointly optimizing its trajectory and resource allocation, and task decision and bits scheduling of users considering fairness. The problem is formulated as a mix-integer nonlinear programming problem with strongly coupled variants, and further transformed into three more tractable subproblems: 1) trajectory optimization $\mathbf{P_T}$; 2) task decision and bits scheduling $\mathbf{P_S}$; and 3) resource allocation $\mathbf{P_R}$. Then, we propose an iterative algorithm to deal with them in a sequence, and further design a penalty method-based algorithm to reduce computation complexity when the branch-and-bound (B&B) algorithm incurs a high complexity to solve $\mathbf{P_S}$. Simulation results demonstrate that our proposed algorithm can efficiently reduce the energy consumption of UAV, and help save 17.7% – 54.6% and 78.9% – 91.9% energy compared with Equal Resource Allocation and Random Resource Allocation. Moreover, it reduces more than 88% running time and achieves relatively satisfactory performance compared with B&B.**

***Index Terms*—UAV, fairness, trajectory, task decision, bits scheduling, resource allocation.**

I. Introduction

With the rapid development of Internet of Things (IoT) and large-scale 5G commercialization, many computation-intensive and latency-sensitive applications, including face recognition, online virtual reality (VR), autonomous vehicles, etc., are increasingly deployed on the resource-limited IoT terminal devices (TDs) [1]. The computational gap between the computation-intensive applications and limited resources of TDs, however, greatly hinders the wide deployment of these novel applications. As the response to this dilemma, Mobile Edge Computing (MEC), an emerging computation paradigm, is put forward to provide more powerful computation capability in proximity to TDs at the network edge. Then, IoT TDs can offload computation-intensive applications to a nearby MEC server to save energy and prolong battery life. Besides offloading, MEC can also be served as a data cache for temporarily storing some frequently accessed data, thereby significantly reducing the response latency and alleviating the traffic of core networks [2]–[5].

Nevertheless, some rural areas lack good network infrastructures, or in some disasters, like an earthquake, the services of network infrastructures may be interrupted. In such cases, unmanned aerial vehicles (UAVs) have been adopted as the aerial MEC server to provide information services in disaster-response, public-safety, and border-monitoring scenarios due to their excellent mobility, flexibility, and cost-efficiency [6]. As a feasible architecture for serving resource-shortage areas, UAV-enabled MEC has attracted much attention in recent years. In the UAV-enabled MEC paradigm, through dynamically adjusting its trajectory, a UAV could be simply adopted as the aerial base station or communication relay for enabling long-range wireless transmission or Internet access [7], [8]. In more commonly-adopted scenarios, a UAV-enabled MEC provides both computation and caching services to multiple TDs, called ground nodes (GNs). A GN could offload its tasks to the MEC-server-carried UAV for processing or caching to reduce latency and alleviate storage pressure whiling meeting specific Quality of Service (QoS) requirements [9]–[12].

However, the operational time of an electric-powered UAV is heavily constrained by its built-in battery, and UAVs usually fly less than an hour or, in some cases, a quarter-hour. If some computation-intensive and memory-hungry tasks are executed on it, it will further shorten the operational time [13]. Especially in the single-UAV scenario, the UAV is more sensitive to energy consumption, since it is the single point and its failure could shut down the whole service. Therefore, energy conservation at the UAV is one of the top priorities for us to design an efficient resource allocation and scheduling algorithm. In this paper, we aim to minimize the UAV's energy consumption by jointly optimizing the UAV's trajectory and resource allocation, and the task decision and bits scheduling

Manuscript received March 14, 2021; revised May 13, 2021, June 12, 2021, and June 30, 2021; accepted July 1, 2021. Date of publication July 6, 2021; date of current version November 22, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 61801418 and Grant 61902341; in part by Yunnan Applied Basic Research Projects under Grant 2019FD-129, Grant 202001BB050034, and Grant 202101AT070182; and in part by the Open Foundation of Key Laboratory in Software Engineering of Yunnan Province under Grant 2020SE316. The editor coordinating the review of this article was I. Krikidis. *(Corresponding author: Di Liu.)*

Mingxiong Zhao and Di Liu are with the National Pilot School of Software and the Key Laboratory in Software Engineering of Yunnan Province, Yunnan University, Kunming 650091, China (e-mail: jimmyzmx@gmail.com; dliu@ynu.edu.cn).

Wentao Li, Lingyan Bao, Jia Luo, and Zhenli He are with the National Pilot School of Software, Yunnan University, Kunming 650091, China (e-mail: lwt@mail.ynu.edu.cn; baolingyan0318@163.com; jialuo_work@163.com; hezl@ynu.edu.cn).

Digital Object Identifier 10.1109/TGCN.2021.3095070

2473-2400 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

of users with fairness concerned. The problem is formulated as a mix-integer nonlinear programming (MINLP) problem with strongly coupled variants. Due to its intractability, we transform the MINLP problem into three more tractable subproblems: 1) trajectory optimization $\mathbf{P_T}$; 2) task decision and bits scheduling $\mathbf{P_S}$; and 3) resource allocation at UAV $\mathbf{P_R}$. Then, we propose an iterative algorithm to deal with them in a sequence. When solving $\mathbf{P_S}$, it incurs a high complexity, so we further design a penalty method (PM)-based algorithm to reduce the computational complexity of solving $\mathbf{P_S}$ instead of the branch-and-bound (B&B) algorithm.

The main contributions of this paper are summarized as follows.

- Considering the tradeoff of energy consumption between computing and caching, a fairness-aware task scheduling and resource allocation scheme is proposed to ensure fairness among different offloading services.
- Since the energy consumption for computing is more than that for caching, a lower bound of the total bits executed for computation is given. It can guarantee fairness among different operations, such as local computing, task offloading for computing or caching, and avoid the situation that the UAV caches all the offloaded bits. Moreover, a lower bound and an upper bound for the scheduled bits within each slot at each GN are further presented to ensure fairness among GNs, prevent the UAV from overwhelming by the numerous data from one or some GNs, and allow GNs to access at any time based on the computational capability and cache capacity of UAV.
- An iterative algorithm with double-loop is derived by leveraging the Successive Convex Approximation (SCA) and Block Coordinate Descent (BCD) methods to solve the formulated MINLP problem. Moreover, due to the high complexity of the B&B algorithm in the inner-loop, a low-complexity algorithm for $\mathbf{P_S}$ is proposed based on **PM** and the equivalent conversion of integer variables, which can save more than 88% running time and achieve relatively satisfactory performance compared with B&B. Simulation results show that our proposed algorithm can save 17.7% – 54.6% and 78.9% – 91.9% energy consumption compared with Equal Resource Allocation and Random Resource Allocation, respectively.

The rest of the paper is organized as follows. Section II summarizes the related work. Section III presents the system model and problem formulation. Section IV describes the proposed algorithm. Section V proposes an alternative algorithm based on the penalty method for task offloading decision subproblem. Section VI gives the numerical results. Finally, Section VII briefly concludes this work.

II. Related Work

In recent years, the UAV-enabled MEC has attracted much attention due to the superior performance of MEC [1], as well as the excellent mobility, flexibility, and cost-efficiency of UAVs [6]. Similar to the terrestrial base station, an MEC-server-mounted UAV could assist GNs in task processing and reduce the task delay to meet the QoS requirements of applications [11], [12]. In [14], Wang *et al.* maximized the number of served tasks according to the hover locations and corresponding communication range of mobile edge servers. Zhang and Ansari utilized the UAV as the computing and relaying node to decrease the average task latency of all users, which jointly considered the user association, resource allocation, and the UAV placement problem [11]. Hu *et al.* studied a joint offloading ratio and user scheduling algorithm to minimize the sum of maximum delay among all users in each time slot in a UAV-enabled system [12]. Employing UAV-enabled MEC servers, IoT-TDs could offload resource-intensive tasks to a nearby server, thus considerably reducing energy consumption and increasing their operational time [9], [10], [15]–[17]. Diao *et al.* used a fixed-wing UAV receiving the GNs' task data and jointly optimized the UAV's trajectory and the task scheduling to minimize the maximum energy consumption among all GNs [9]. Hu *et al.* minimized the total GNs' energy consumption with QoS requirements by setting UAV positions as a globally optimal result of two-dimensional search [10], while de Silva and da Fonseca minimize GNs' energy consumption via optimizing the location of fog nodes [16]. Li *et al.* proposed a collaborative offloading and caching strategy to minimize the total energy consumption of GNs [17].

Although UAV-enabled MEC servers facilitate flexible computation and storage in some special areas or situations, UAVs that are battery-supplied suffer from the short flying time issue. Therefore, several studies have been devoted to optimize the UAV's energy consumption by optimizing the resource allocation at MEC servers [18]–[21], task scheduling and offloading [13], [19], [20], [22], as well as the UAV's trajectory. Du *et al.* aimed to minimize the UAV's energy cost by optimizing the UAV's resource allocation and its hovering strategy [18]. Hua *et al.* considered the power allocation and bit allocation with different access strategies in a multi-UAV enabled system [19]. Mei *et al.* optimized the resource allocation of UAVs by solving a series of convex quadratically constrained programming problems [21]. An iterative algorithm was derived to solve the discrete offloading problem in [13], which jointly considered the resource allocation and the UAV's trajectory optimization. Liu *et al.* designed iterative algorithms, based on the SCA technique, to jointly optimize the task scheduling, computing, and transmitting resource allocation [22]. The weighted energy consumption of the UAV and GNs was jointly considered in [23]–[29]. Zhang *et al.* proposed an iterative algorithm to minimize the energy consumption of task transmitting, task computing, and the UAV's propulsion by jointly optimizing the power and bits allocation, the UAV's trajectory and slot schedule [26]. Zhang *et al.* put forward the system utility of the overall weighted energy consumption of multi-UAV enabled MEC networks and alternately solved the formulated problem by leveraging the successive convex programming and quadratic approximation techniques [27]. Some close works are summarized in Table I.

To the best of our knowledge, few efforts have been devoted to the fairness-aware offloading scheme with the consideration of different operations, such as local computing, task offloading for computing, or caching according to the specific need in the UAV-enabled MEC networks. Compared with previous studies, this work considers both latency and energy

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF THE RELATED WORKS

Offloading Service	Objective Metrics	Model	Access Model	Propulsion Model	Offloading Scheme	Fairness Consideration among Offloading Services and GNs	Article
Computing Offloading	Task Latency	LoS	TDMA	$E_{\mathrm{uav}}=\sum \kappa \lVert v \rVert^2$	Binary	-	[12]
		LoS&NLoS	TDMA	-		-	[11]
	GN Energy Consumption	LoS	OFDMA	Fixed-wing	-	GNs Energy Consumption Fairness	[9]
			TDMA	-		-	[10]
			FDD	-	Partial	-	[15]
	UAV Energy Consumption	LoS	TDMA	Fixed-wing	Binary	-	[13]
			TDMA		Partial	-	[22]
			FDMA	$E_{\mathrm{uav}}=P^{\mathrm{hover}}\times T$	-	-	[18]
			TDMA			-	[20]
		LoS&NLoS	OFDMA	Rotary/Fixed-wing		-	[21]
	Tradeoff between UAV and GN Energy Consumption	LoS	TDMA	Fixed-wing	Partial	-	[19]
			OFDMA	Rotary-wing		-	[26]
			FDD	Fixed-wing		-	[24]
			TDMA	$E_{\mathrm{uav}}=\sum \kappa \lVert v \rVert^2$	Binary	-	[23]
			FDMA		Partial	-	[25]
			FDMA			-	[28]
			TDMA		Binary	-	[29]
		LoS&NLoS	NOMA		Partial	-	[27]
Hybrid Offloading	GN Energy Consumption	LoS	FDMA	Rotary-wing		-	[17]
Hybrid Task Scheduling	UAV Energy Consumption					GNs & Task Fairness	This work

requirements, and the fairness issues in resource allocation, task decision, and bits scheduling problems. In the meantime, according to the equivalent conversion of integer variables, we provide an alternative algorithm based on the penalty method to replace the traditional B&B algorithm for solving the discrete binary integer programming problem.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this paper, we consider a UAV-enabled MEC network based on frequency division multiple access, where multiple GNs are denoted by $\mathcal{K}=\{1,2,3,\ldots,K\}$ and randomly distributed on the ground, i.e., $\mathbf{w}_k=(x_k,y_k)^T,\ \forall k$. Define the whole time of navigation as *T*, and let $\mathbf{Q}_0$ and $\mathbf{Q}_F$ denote the origin and the final point in the 3D Cartesian coordinate system, respectively. The UAV flies at the horizontal plane of fixed altitude *H*, and provides data computing and caching services for resource-limited GNs during its navigation, while GNs remain stationary. The discrete UAV trajectory model is adopted similar to [30], [31], where *T* is partitioned into *N* slots and each is with the length of $\tau=T/N$ (in second). Denote $\boldsymbol{q}[n]=(x[n],y[n])^T$ as the position of UAV at the *n*-th slot, and indicate $v_{\max}$ as the maximal speed for UAV. Since τ is small, the average speed between $\boldsymbol{q}[n+1]$ and $\boldsymbol{q}[n]$ can be approximated by the instant speed $\lVert \boldsymbol{v}[n]\rVert$, given by

$$v[n]\triangleq\lVert \boldsymbol{v}[n]\rVert=\frac{\lVert \boldsymbol{q}[n+1]-\boldsymbol{q}[n]\rVert}{\tau}, \tag{1}$$

where $\lVert\cdot\rVert$ denotes the Euclidean distance, and *v*[*n*] cannot exceed its maximum speed $v_{\max}$, $\lVert \boldsymbol{q}[n+1]-\boldsymbol{q}[n]\rVert\le\tau v_{\max}$. Besides, the distance between the UAV and the *k*-th GN within the *n*-th slot can be written as

$$d_k[n]=\sqrt{\lVert \boldsymbol{q}[n]-\mathbf{w}_k\rVert^2+H^2}. \tag{2}$$

For convenience, the notations used in this paper are summarized in Table II.

TABLE II
SUMMARY OF NOTATIONS

Symbols	Definitions
K	the number of devices
N	the number of slots
$\mathbf{w}_k$	the position of the k-th GN
$\boldsymbol{q}[n]$	the trajectory of UAV at the n-th slot
$d_k[n]$	the distance between the k-th GN and UAV
$h_k[n]$	the channel gain between the k-th GN and UAV
β_0	the channel gain at the reference distance 1m
$b_k[n]$	the bandwidth proportion of the k-th GN
H	the flight height of UAV.
C	the cache capacity of UAV
B	the total bandwidth of UAV
F	the total computation capability of UAV
p_k	the transmission power of the k-th GN
f_k	the local CPU frequency of the k-th GN
E_k	the energy budget of the k-th GN
T_k	the latency requirement of the k-th GN
S_k	the total bits to be processed of the k-th GN
S_k^{exe}	the total bits executed for computation at the UAV or at the k-th GN
$s_k[n]$	the scheduled bits of the k-th GN at the n-th slot
$r_k[n]$	the achievable rate of the k-th GN
$f_k^u[n]$	the computation frequency assigned by UAV to the k-th GN
$a_k^1[n]$	the local computing indicator
$a_k^2[n]$	the remote computing indicator
$a_k^3[n]$	the remote caching indicator
ω	the caching energy coefficient
τ	the length of slot
η	the computing energy coefficient at the UAV
η_k	the computing energy coefficient at the k-th GN
θ	the CPU cycles for executing 1 bit at the UAV
θ_k	the CPU cycles for executing 1 bit at the k-th GN
$v[n]$	the speed of UAV flight
$o[n],\tilde{r}^q[n],\tilde{r}^b[n]$	the auxiliary variables

A. Transmission Model

Assume that the wireless channel between the UAV and the *k*-th GN is dominated by the line-of-sight (LoS) link, modeled by the free space path loss model [26], [32]. Denote the channel gain from the *k*-th GN to the UAV at the *n*-th slot as $h_k[n]$, given by

$$h_k[n]=\frac{\beta_0}{d_k[n]^2}=\frac{\beta_0}{\lVert \boldsymbol{q}[n]-\mathbf{w}_k\rVert^2+H^2}, \tag{3}$$

where β_0 denotes the channel gain at the reference distance d_0 = 1 m. The total bandwidth *B* is shared among GNs, the proportion of which assigned to the *k*-th GN is $b_k[n]$, and must satisfy

$$\sum_{k=1}^{K} b_k[n]\le 1,\forall n. \tag{4}$$

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

Therefore, the achievable rate of the k-th GN in bit-per-second (bps) can be written as

$$\begin{aligned} r_k[n] &= Bb_k[n]\log_2\left(1+\frac{p_k h_k[n]}{N_0 Bb_k[n]}\right) \\ &= Bb_k[n]\log_2\left(1+\frac{p_k\phi_0}{\|\boldsymbol{q}[n]-\mathbf{w}_k\|^2+H^2}\right), \end{aligned} \tag{5}$$

where p_k is the transmit power of the k-th GN. $\phi_0 = \frac{\beta_0}{N_0 Bb_k[n]}$ denotes the reference received signal-to-noise ratio, where N_0 is the noise power spectrum density at the UAV.

B. Task Execution Model

During the flight, each GN runs the main loop application generating many bits, which can be processed concerning different operations such as local computing, task offloading for computing, or caching according to the specific need, where we assume that GN does not store data at its local buffer. Due to the limited computational capability and energy supply at GNs, indicated by f_k and E_k, $\forall k$, these bits can be offloaded to UAV for further processing instead of computing locally to free GNs from computation-intensive or energy-consuming workloads. Denote S_k and $s_k[n]$ as the total number of bits at the k-th GN and its amount of bits scheduled at the n-th slot, respectively, which satisfies $\sum_{n=1}^{N} s_k[n] = S_k,\ \forall k$. With the consideration of fairness and stability for networks, there are a lower bound and an upper bound for the scheduled bits within each slot for each GN,

$$S_k^{\min} \le s_k[n] \le S_k^{\max},\ \forall k, n, \tag{6}$$

where $S_k^{\min}$ represents the minimum generated bits to be processed ensuring the fairness among GNs, and $S_k^{\max}$ indicates the maximal reasonable workload to be scheduled according to the computational capability and cache capacity at the UAV shared among GNs, making the UAV-enabled MEC network more stable.[1] Moreover, to guarantee successful offloading, a lower bound for the channel capacity between UAV and the k-th GN is given as

$$r_k[n] \ge R_k^{\min}, \forall k, n. \tag{7}$$

Denote a ternary variable $a_m^k[n],\ m = \{1, 2, 3\}$ as the decision of scheduled bits at the k-th GN, where $a_1^k[n] = 1$ means local computation, $a_2^k[n] = 1$ stands for computation offloading and $a_3^k[n] = 1$ represents offloading for caching. Since only one operation can be performed at a time, the following constraint must be satisfied

$$\sum_{m=1}^{3} a_m^k[n] = 1, \forall k, n. \tag{8}$$

In addition, the total sizes for computation of each GN should satisfy

$$\sum_{n=1}^{N}\Big(a_1^k[n] + a_2^k[n]\Big)s_k[n] \ge S_k^{\text{exe}},\ \forall k, \tag{9}$$

[1]The reason to restrict the maximal bits of GNs in each slot is to guarantee that the offloading request of any GN can be responded, i.e., the UAV cannot be overwhelmed by the numerous data from one or some GNs, and GNs can access to the UAV at any time. Moreover, it can efficiently balance resource allocation among GNs to avoid the saturated performance due to unfair resource distribution.

to guarantee fairness among different operations of services. Since the energy consumption for computing is more than that for caching, we add this constraint to avoid the situation that the UAV caches all the offloaded bits, where S_k^{exe} represents the total bits executed for computation at the UAV or the k-th GN. Meanwhile, buffer space will be assigned if caching is performed, and the total cached bits cannot exceed the cache capacity of UAV, given by

$$\sum_{n=1}^{N}\sum_{k=1}^{K} a_3^k[n]s_k[n] \le C, \tag{10}$$

where C is the maximal cache capacity of UAV. If the contents or tasks were stored, they can be used by other TDs without the extra transmission in the future [33], and provide content transfer services for users to meet their frequent requests. Moreover, the usage of caching can also help to buffer the abundant data or tasks until the UAV has sufficient computation resources. Since UAV can process the cached data instead, computation capability at the UAV will not be wasted in the situation where a GN has the low-rate transmission due to poor channel gain [17]. Thus, the UAV can have the cache to save tasks or contents in its storage area to relieve the backhaul bottleneck, reduce the response latency [34], [35], and foster efficiency in the use of communication resource and computation capability concurrently [17].

If the scheduled bits are executed locally, the processing delay at GNs can be written as

$$t_k^{\text{loc}}[n] = a_1^k[n]\frac{s_k[n]\theta_k}{f_k},\ \forall k, n, \tag{11}$$

where θ_k denotes the CPU computation intensity at the k-th GN. If offloaded to the UAV, the offloading delay should be considered, including transmission delay and processing delay. Since the time for cache writing is negligible, the processing delay is ONLY for computing task [21], [28]. Therefore, the offloading delay is given as

$$\begin{aligned} t_k^{\text{off}}[n] &= t_k^{\text{com}}[n] + t_k^{\text{up}}[n] \\ &= a_2^k[n]\frac{s_k[n]\theta}{f_k^u[n]} + (1-a_1^k[n])\frac{s_k[n]}{r_k[n]}, \end{aligned} \tag{12}$$

where $t_k^{\text{com}}[n]$ denotes the computing delay, and $t_k^{\text{up}}[n]$ represents the uplink transmission delay, and θ denotes the CPU cycles required for computing 1-bit at the UAV. Furthermore, the total allocated CPU frequency cannot exceed the total computation capability of UAV

$$\sum_{n=1}^{K} a_2^k[n]f_k^u[n] \le F, \tag{13}$$

where $f_k^u[n]$ stands for the computation frequency assigned by the UAV to the k-th GN. To ensure the instantaneous performance of the network, the processing delay of the scheduled bits at each slot should be constrained

$$\begin{aligned} t_k[n] &\triangleq \max\Big\{t_k^{\text{loc}}[n], t_k^{\text{off}}[n]\Big\} \\ &= t_k^{\text{loc}}[n] + t_k^{\text{off}}[n] \le \tau,\ \forall k, n, \end{aligned} \tag{14}$$

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

where the second equality holds since ONLY one operation is allowed at a clip referring to (8), i.e., $\sum_{m=1}^{3} a_m^k[n] = 1, \forall k, n$, where only one element can be 1. Moreover, due to the latency requirements of applications, the application at the k-th GN should be completely executed within the given time

$$\sum_{n=1}^{N} t_k[n] \leq T_k, \tag{15}$$

where $T_k \leq T,\ \forall k$.

C. Energy Consumption Model

1) Energy Consumption at GNs: Given the processor's computing capability f_k, the power consumption of the processor is modeled as $\eta_k f_k^3$ (joule per second), where κ_k represents the computation energy efficiency coefficient related to the processor's chip equipped at the k-th GN [36]–[38]. Taking the processing delay (11) into consideration, the energy consumption for local computing is given by

$$E_k^{\text{loc}}[n] = \eta_k f_k^3 t_k^{\text{loc}}[n] = a_1^k[n] s_k[n] \theta_k \eta_k f_k^2. \tag{16}$$

If offloaded to the UAV, the energy consumption for transmission at the k-th GN is given as

$$E_k^{\text{tran}}[n] = p_k t_k^{\text{up}}[n] = (1 - a_1^k[n]) p_k \frac{s_k[n]}{r_k[n]}. \tag{17}$$

Moreover, the total energy consumption of the k-th GN cannot exceed its energy budget

$$\sum_{n=1}^{N} E_k^{\text{tran}}[n] + E_k^{\text{loc}}[n] \leq E_k, \forall k. \tag{18}$$

2) Energy Consumption at the UAV: During the offloading process, the UAV allocates computing resource or cache space for these offloaded bits. Similarly, for computation offloading, the energy consumption for executing the bits of the k-th GN is given as

$$E_k^{\text{exe}}[n] = a_2^k[n] s_k[n] \theta \eta f_k^u[n]^2, \tag{19}$$

where η is the CPU energy consumption coefficient of the UAV [27], [38]. Moreover, the UAV can also cache the offloaded bits temporarily with the purpose of saving energy or avoiding CPU overload during the flight. According to the energy-proportional model [39], the energy consumption for caching the bits from the k-th GN is given by

$$E_k^{\text{cac}}[n] = a_3^k[n] \omega s_k[n], \tag{20}$$

where ω is the caching energy coefficient related to the storage hard-drive of UAV [40]. Furthermore, similar to [30], [31], the rotary-wing energy model is adopted for UAV propulsion energy consumption, given by

$$E_{\text{fly}} = \sum_{n=1}^{N} \tau \left[P_0 \left(1 + \frac{3v^2[n]}{Q_{\text{tip}}^2} \right) + P_C v^3[n] + P_i \left(\sqrt{1 + \frac{v^4[n]}{4v_0^4}} - \frac{v^2[n]}{2v_0^2} \right)^{\frac{1}{2}} \right], \tag{21}$$

where P_0 and P_i denote the constant *blade profile power* and *induced power* in hovering status. Q_{tip} is the tip speed of the rotor blade, v_0 represents the mean rotor induced speed in hover, and P_C is a constant value related to environment. Thus, the total energy consumption of UAV can be written as

$$E_{\text{uav}} = E_{\text{fly}} + \sum_{k=1}^{K} \sum_{n=1}^{N} E_k^{\text{cac}}[n] + \sum_{k=1}^{K} \sum_{n=1}^{N} E_k^{\text{exe}}[n]. \tag{22}$$

For notation simplicity, we define $\boldsymbol{Q} \triangleq \{\boldsymbol{q}[n]\}$, $\boldsymbol{A} \triangleq \{\boldsymbol{A}_1, \boldsymbol{A}_2, \boldsymbol{A}_3\}$ where $\boldsymbol{A}_m \triangleq \{a_m^k[n]\}$, $\boldsymbol{S} \triangleq \{s_k[n]\}$, $\boldsymbol{B} \triangleq \{b_k[n]\}$, $\boldsymbol{F} \triangleq \{f_k^u[n]\}$, $\forall k, n, m$.

D. Problem Formulation

In this paper, we aim to minimize the energy consumption of UAV by jointly optimizing the trajectory of UAV $\boldsymbol{Q}$, task decision $\boldsymbol{A}$ and bits scheduling $\boldsymbol{S}$, bandwidth $\boldsymbol{B}$ and computation resource allocation $\boldsymbol{F}$. Accordingly, the optimization problem can be written as

$$\mathbf{P}: \min_{\{\boldsymbol{Q}, \boldsymbol{A}, \boldsymbol{S}, \boldsymbol{B}, \boldsymbol{F}\}} \quad E_{\text{uav}}$$

$$\text{s.t.} \quad \sum_{n=1}^{N} s_k[n] = S_k,\ \forall k, \tag{23a}$$

$$S_k^{\min} \leq s_k[n] \leq S_k^{\max}, \forall k, n, \tag{23b}$$

$$\boldsymbol{q}[0] = \mathbf{Q}_0,\ \boldsymbol{q}[N] = \mathbf{Q}_F, \tag{23c}$$

$$\|\boldsymbol{q}[n+1] - \boldsymbol{q}[n]\| \leq v_{\max}\tau, \forall n, \tag{23d}$$

$$0 \leq v[n] \leq v_{\max}, \forall n, \tag{23e}$$

$$0 \leq f_k^u[n] \leq F, \forall k, n, \tag{23f}$$

$$b_k[n] \in [0, 1],\ \forall k, n, \tag{23g}$$

$$\sum_{k=1}^{K} b_k[n] \leq 1, \forall n, \tag{23h}$$

$$r_k[n] \geq R_k^{\min}, \forall k, n, \tag{23i}$$

$$\sum_{m=1}^{3} a_m^k[n] = 1, \forall k, n, \tag{23j}$$

$$\sum_{n=1}^{N} \left(a_1^k[n] + a_2^k[n] \right) s_k[n] \geq S_k^{\text{exe}}, \forall k, \tag{23k}$$

$$\sum_{n=1}^{N} \sum_{k=1}^{K} a_3^k[n] s_k[n] \leq C, \tag{23l}$$

$$\sum_{n=1}^{K} a_2^k[n] f_k^u[n] \leq F, \tag{23m}$$

$$a_m^k[n] \in \{0, 1\}, \forall k, m, n, \tag{23n}$$

$$t_k^{\text{loc}}[n] + t_k^{\text{off}}[n] \leq \tau, \forall k, n, \tag{23o}$$

$$\sum_{n=1}^{N} t_k[n] \leq T_k, \tag{23p}$$

$$\sum_{n=1}^{N} E_k^{\text{tran}}[n] + E_k^{\text{loc}}[n] \leq E_k, \forall k, \tag{23q}$$

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

where constraint (23i) ensures that successful offloading can be performed based on the channel conditions, (23k) guarantees the fairness among different operations, constraints (23o) and (23p) are the latency requirements, while constraints (23l), (23m) and (23q) are for task scheduling and resource allocation. Notice that the above-formulated problem is an MINLP problem, which is highly intractable due to the ternary variable in constraint (23j) and the strongly coupled variables in the objective function and constraints. Moreover, these non-linear constraints (23o)-(23q) coupled with the integer variables further make $\mathbf{P}$ harder.

IV. Proposed Algorithm

Since the problem $\mathbf{P}$ is an MINLP problem and $\{\boldsymbol{Q}, \boldsymbol{A}, \boldsymbol{S}, \boldsymbol{B}, \boldsymbol{F}\}$ are still coupled in both the objective function and constraints, it is impossible to directly obtain the solution of $\mathbf{P}$ in its current form. Motivated by the iterative design, we employ the BCD method to decouple these variables in $\mathbf{P}$, due to its benefit of tackling multiple coupled variables. Therefore, we divide the original problem into three more manageable subproblems: 1) $\mathbf{P_T}$: UAV trajectory optimization $\boldsymbol{Q}$; 2) $\mathbf{P_S}$: task decision and bits scheduling, $\boldsymbol{A}$ and $\boldsymbol{S}$; and 3) $\mathbf{P_R}$: communication and computation resource allocation $\boldsymbol{B}$ and $\boldsymbol{F}$ at the UAV. Then we alternately optimize these subproblems according to the BCD method [41], [42]. The application of the BCD method is given as follows.

1) **Step 1**: given task decision $\boldsymbol{A}$, bits scheduling $\boldsymbol{S}$, communication $\boldsymbol{B}$ and computation resource allocation $\boldsymbol{F}$, we obtain the optimal trajectory $\boldsymbol{Q}^\star$ at the i-th iteration in its inner loop[2];
2) **Step 2**: with the newly obtained trajectory $\boldsymbol{Q}^\star$, we optimize task decision $\boldsymbol{A}$ and bits scheduling $\boldsymbol{S}$ according to a linear programming solver and B&B algorithm in its inner loop iteratively;
3) **Step 3**: employing solver CVX, we derive the optimal $\{\boldsymbol{B}^\star, \boldsymbol{F}^\star\}$ resorting to the auxiliary variable, which makes $\mathbf{P_R}$ convex with the newly obtained $\{\boldsymbol{Q}^\star, \boldsymbol{A}^\star, \boldsymbol{S}^\star\}$;
4) **Step 4**: we will iteratively update the derived $\{\boldsymbol{Q}, \boldsymbol{A}, \boldsymbol{S}, \boldsymbol{B}, \boldsymbol{F}\}$ in the outer loop, and the procedures are known as the BCD method.

In the remainder of this section, we present the details of our proposed method including UAV trajectory optimization, task decision and bits scheduling, and resource allocation. Finally, we introduce the overall algorithm and analyze its complexity.

A. UAV Trajectory Optimization

Given $\{\boldsymbol{A}, \boldsymbol{S}, \boldsymbol{B}, \boldsymbol{F}\}$, the trajectory of UAV $\boldsymbol{Q}^\star$ can be derived from the following problem

$$\mathbf{P_T}\colon \min_{\boldsymbol{Q}} \ E_{\text{uav}}(\boldsymbol{Q}) \quad \text{s.t.} \quad (23\text{c}) - (23\text{e}),\ (23\text{i}),\ (23\text{o}) - (23\text{q}),$$

which is non-convex due to the non linearities lying in the objective function and constraints (23i), (23o) and (23q) with respect to $\boldsymbol{Q}$.

[2]The outer loop is designed for the three subproblems, i.e., $\mathbf{P_T}$, $\mathbf{P_S}$ and $\mathbf{P_R}$, and to update the achieved variants iteratively.

To make $\mathbf{P_T}$ more tractable, the SCA technique is utilized to cope with the non-convexity employing the auxiliary variable $\boldsymbol{o} \triangleq \{o[n] \geq (\sqrt{1+\frac{v^4[n]}{4v_0^4}} - \frac{v^2[n]}{2v_0^2})^{\frac{1}{2}},\ \forall n\}$, and thus make the objective function convex with respect to $v[n]$. Employing the first-order Taylor's expansion at the i-th iteration, $o[n]$ can be further transformed into the following inequality with given points $(o_i[n], v_i[n])$,

$$\frac{1}{o^2[n]} \leq 2o_i[n]o[n] - o_i^2[n] + \frac{v_i[n]}{v_0^2}(2v[n] - v_i[n]). \quad (25)$$

Moreover, the slack variable $\widetilde{\boldsymbol{R}}^q \triangleq \{\tilde{r}_k^q[n] \geq 0, \forall k, n\}$ is introduced to deal with the non-convex parts related with $r_k[n]$ in the constraints with a new constraint, given by

$$\log_2\left(1 + \frac{p_k\phi_0}{\|\boldsymbol{q}[n] - \mathbf{w}_k\|^2 + H^2}\right) \geq \frac{\tilde{r}_k^q[n]}{Bb_k[n]},\ \forall k, n. \quad (26)$$

Although the left-hand side (LHS) of (26) is non-convex with respect to $\boldsymbol{q}[n]$, it is convex with respect to $\|\boldsymbol{q}[n] - \mathbf{w}_k\|$. Recall that the first-order Taylor approximation of a convex function is a global underestimator [13], with a given local point $\boldsymbol{q}_i[n]$ at the i-th iteration, $r_k[n]$ can be lower bounded as

$$\begin{aligned} r_k^i[n] &\triangleq R_k^i[n] + \nabla R_k^i[n](\|\boldsymbol{q}[n] - \mathbf{w}_k\| - \|\boldsymbol{q}_i[n] - \mathbf{w}_k\|) \\ &\geq \frac{\tilde{r}_k^q[n]}{Bb_k[n]},\ \forall k, n, \end{aligned} \quad (27)$$

where $R_k^i[n] = \log_2(1 + p_k\phi_0/d_k^i)$ and $\nabla R_k^i[n] = \frac{-p_k\phi_0}{d_k^i(d_k^i + p_k\phi_0)\ln 2}$ are the constant rate and the first derivative at the i-th iteration, where $d_k^i \triangleq \|\boldsymbol{q}_i[n] - \mathbf{w}_k\|^2 + H^2$. In the end, $\mathbf{P_T}$ can be transformed into the following convex optimization problem

$$\min_{\{\boldsymbol{Q}, \boldsymbol{v}, \boldsymbol{o}, \widetilde{\boldsymbol{R}}^q\}} \ \tau \sum_{n=1}^{N} P_0\left(1 + \frac{3v^2[n]}{Q_{\text{tip}}^2}\right) + P_i o[n] + P_C v^3[n] \quad (28\text{a})$$

$$\mathbf{P_T1}\text{:s.t.} \quad \frac{1}{o^2[n]} \leq 2o_i^2[n]o[n] - o_i^2[n] + \frac{v_i[n]}{v_0^2}(2v[n] - v_i[n]),\ \forall n, \quad (28\text{b})$$

$$a_1^k[n]\frac{s_k[n]\theta_k}{f_k} + a_2^k[n]\frac{s_k[n]\theta}{f_k^u[n]} + (1 - a_1^k[n])\frac{s_k[n]}{\tilde{r}_k^q[n]} \leq \tau,\ \forall k, n, \quad (28\text{c})$$

$$\sum_{n=1}^{N}\left\{a_1^k[n]\frac{s_k[n]\theta_k}{f_k} + a_2^k[n]\frac{s_k[n]\theta}{f_k^u[n]} + (1 - a_1^k[n])\frac{s_k[n]}{\tilde{r}_k^q[n]}\right\} \leq T_k,\ \forall k, \quad (28\text{d})$$

$$\sum_{n=1}^{N}\left\{(1 - a_1^k[n])p_k\frac{s_k[n]}{\tilde{r}_k^q[n]} + a_1^k[n]s_k[n]\theta_k\eta_k f_k^2\right\} \leq E_k, \quad (28\text{e})$$

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Algorithm for Trajectory Optimization

Input: Given initial points $\{\boldsymbol{Q}_i, \boldsymbol{B}, \boldsymbol{F}, \boldsymbol{A}, \boldsymbol{S}\}$, where we set $i = 0$.

Output: $\left\{\boldsymbol{Q}^\star, \boldsymbol{v}^\star, \boldsymbol{o}^\star, \tilde{\boldsymbol{R}}^{q,\star}\right\}$.

1: **Initialization:** $v_i[n] = \|\boldsymbol{q}_i[n+1] - \boldsymbol{q}_i[n]\|/\tau,\ o_i[n] = \left(\sqrt{(1+\frac{\boldsymbol{v}_i[n]^4}{4v_0^4})} - \frac{\boldsymbol{v}_i[n]^2}{2v_0^2}\right)^{\frac{1}{2}}$ and $\tilde{\boldsymbol{R}}_i^q[n] = \{\tilde{\boldsymbol{r}}_i^q[n]\}, \forall n$, where $\tilde{\boldsymbol{r}}_i^q[n] \triangleq \{\tilde{r}_{k,i}^q[n], \forall k\}$.

2: **repeat**

3: Given local points $\{\boldsymbol{q}_i[n], o_i[n], v_i[n], \tilde{r}_{k,i}^q[n], \forall k\}$, we can further obtain the optimal solutions $\left\{\boldsymbol{Q}^\star[n], \boldsymbol{o}^\star[n], \boldsymbol{v}^\star[n], \tilde{\boldsymbol{R}}^{q,\star}[n]\right\}$ by solving $\mathbf{P_T1}$.

4: Update local points $\{\boldsymbol{q}_{i+1}[n] = \boldsymbol{q}^\star[n], o_{i+1}[n] = o^\star[n], v_{i+1}[n] = v^\star[n], \tilde{r}_{k,i+1}^q[n] = \tilde{r}_q^\star[n]\}$.

5: **until** Converge to the tolerated accuracy.

$$\|\boldsymbol{q}[n] - \mathbf{w}_k\|^2 + H^2 \le \frac{\phi_0}{2^{\frac{R_k^{\min}}{Bb_k[n]}} - 1},\ \forall k, n, \tag{28f}$$

$$R_k^i[n] + \nabla R_k^i[n](\|\boldsymbol{q}[n] - \mathbf{w}_k\|^2 - \|\boldsymbol{q}_i[n] - \mathbf{w}_k\|^2) \ge \frac{\tilde{r}_k^q[n]}{Bb_k[n]},\ \forall k, n,$$
$$(23c) - (23e), \tag{28g}$$

where $\boldsymbol{v} \triangleq \{v[n], \forall n\}$. $\mathbf{P_T1}$ can be solved by solvers, e.g., CVX [43], and the details can be referred to Algorithm 1.

B. Task Decision and Bits Scheduling

With the optimized trajectory $\boldsymbol{Q}^\star$ and $\{\boldsymbol{B}, \boldsymbol{F}\}$, task decision and bits scheduling $\{\boldsymbol{A}, \boldsymbol{S}\}$ can be derived by solving the following optimization problem to minimize the energy consumption of UAV

$$\mathbf{P_S}\colon \min_{\{\boldsymbol{A},\boldsymbol{S}\}} \sum_{n=1}^{N}\sum_{k=1}^{K} a_2^k[n]s_k[n]\theta\eta f_k^u[n]^2 + a_3^k[n]\omega s_k[n]$$
$$\text{s.t.}\quad (23a),\ (23b),\ (23j) - (23q), \tag{29a}$$

which is also non-convex due to the binary variable $\boldsymbol{A}$ and the highly coupled variants ($\boldsymbol{A}$ and $\boldsymbol{S}$) in both the objective function and the constraints. To decouple these variables, a BCD-based algorithm is proposed to divide $\mathbf{P_S}$ into two subproblems as follows:

- ***A linear programming (LP) problem*** with respect to bits scheduling $\mathbf{S}$, which can be solved by linear programming solvers.
- ***An integer linear programming (ILP) problem*** with respect to task decision $\boldsymbol{A}$, which can be solved by B&B. By constructing the search tree and calculating the boundary value, B&B can obtain the exact solutions through recursively traversing the solution space [3].

Accordingly, the optimal solutions for bits scheduling and task decision can be derived via solving $\mathbf{P_S}$ in an alternative manner. To be specific, with $\boldsymbol{S}_{j-1}$, $\boldsymbol{A}_j^\star$ can be optimized via LP at the *j*-th iteration. Then we can further solve ILP to obtain $\boldsymbol{S}_j^\star$ with the newly obtained $\boldsymbol{A}_j^\star$ at the *j*-th iteration. The iterative process will terminate when the stopping criteria are satisfied.

C. Resource Allocation

Based on the newly derived $\{\boldsymbol{Q}^\star, \boldsymbol{A}^\star, \boldsymbol{S}^\star\}$, communication bandwidth and computation resource allocation can be obtained by solving $\mathbf{P_R}$ to further minimize energy consumption of UAV in this subsection

$$\mathbf{P_R}\colon \min_{\{\boldsymbol{B},\boldsymbol{F}\}} \sum_{n=1}^{N}\sum_{k=1}^{K} a_2^k[n]s_k[n]\theta\eta f_k^u[n]^2$$
$$\text{s.t.}\quad (23f) - (23i),\ (23m),\ (23o) - (23q). \tag{30a}$$

Since the constraints related with latency requirements (23o), (23p) and energy supply (23q) are non-convex and non-linear due to the term "$r_k[n]$", the auxiliary variable $\tilde{\boldsymbol{R}}^b \triangleq \{\tilde{r}_k^b[n] \ge 0, \forall k, n\}$ is introduced to deal with these strongly coupled constraints

$$r_k[n] = Bb_k[n]\log_2\left(1 + \frac{\phi_1}{Bb_k[n]}\right) \ge \tilde{r}_k^b[n], \forall k, n, \tag{31}$$

where $\phi_1 \triangleq \frac{p_k h_k[n]}{N_0}$ and the LHS of (31) is concave with respect to $b_k[n]$. Notice that the second order derivative of the LHS in constraint (31) is given as $\nabla^2 r_k[n](b_k[n]) = -\frac{B\phi_1^2}{b_k[n](Bb_k[n]+\phi_1)^2\ln 2}$, which proves the concavity of constraint (31) referring to [44].

Replacing $r_k[n]$ with $\tilde{r}_k^b[n]$ and adding constraint (31) into $\mathbf{P_R}$, it can be transformed into the following problem

$$\min_{\{\boldsymbol{B},\boldsymbol{F},\tilde{\boldsymbol{R}}^b\}} \sum_{n=1}^{N}\sum_{k=1}^{K} E_{\text{uav}}(\boldsymbol{B}, \boldsymbol{F}) \tag{32a}$$

$$\mathbf{P_R1}\colon \text{s.t.}\quad \frac{a_1^k[n]s_k[n]\theta_k}{f_k} + \frac{a_2^k[n]s_k[n]\theta}{f_k^u[n]} + \frac{(1-a_1^k[n])s_k[n]}{\tilde{r}_k^b[n]} \le \tau, \forall k, n, \tag{32b}$$

$$\sum_{n=1}^{N}\left\{a_1^k[n]s_k[n]\theta_k\eta_k f_k^2 + \frac{(1-a_1^k[n])p_k s_k[n]}{\tilde{r}_k^b[n]}\right\} \le E_k, \forall k, \tag{32c}$$

$$\sum_{n=1}^{N}\left\{\frac{a_1^k[n]s_k[n]\theta_k}{f_k} + \frac{a_2^k[n]s_k[n]\theta}{f_k^u[n]} + \frac{(1-a_1^k[n])s_k[n]}{\tilde{r}_k^b[n]}\right\} \le T_k, \forall k, \tag{32d}$$

$$Bb_k[n]\log_2\left(1 + \frac{\phi_1}{Bb_k[n]}\right) \ge R_k^{\min}, \forall k, n,$$
$$(23f),\ (23g),\ (23h),\ (23m),\ (31), \tag{32e}$$

where $\mathbf{P_R1}$ is a convex problem with respect to $\{\boldsymbol{B}, \boldsymbol{F}, \tilde{\boldsymbol{R}}^b\}$ according to [44], and it can be efficiently solved by solvers, e.g., CVX [43].

D. Overall Algorithm

The suboptimal solution of the original problem can be derived by alternately optimizing subproblems $\mathbf{P_T}, \mathbf{P_S}$ and $\mathbf{P_R}$ in an iterative manner until the tolerated accuracy is achieved

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Proposed Overall Algorithm

Input: Given initial points $\{\boldsymbol{Q}_i, \boldsymbol{B}_i, \boldsymbol{F}_i, \boldsymbol{A}_i, \boldsymbol{S}_i\}$, we set $i = 0$.

Output: $\{\boldsymbol{Q}^\star, \boldsymbol{B}^\star, \boldsymbol{F}^\star, \boldsymbol{A}^\star, \boldsymbol{S}^\star\}$.

1: **repeat**
2: Given $\{\boldsymbol{B}_i, \boldsymbol{F}_i, \boldsymbol{A}_i, \boldsymbol{S}_i\}$, we optimize the trajectory of UAV $\boldsymbol{Q}_{i+1}$ by solving $\mathbf{P_T1}$ based on Algorithm 1.
3: Given $\{\boldsymbol{B}_i, \boldsymbol{F}_i\}$ and the newly derived $\boldsymbol{Q}_{i+1}$, the task decision and bits scheduling $\{\boldsymbol{A}_{i+1}, \boldsymbol{S}_{i+1}\}$ can be obtained as follows (From Step 4 to Step 8), where we initialize $j = 0$ in the inner loop for $\mathbf{P_S}$.
4: **repeat**
5: Solve the **LP** problem to obtain $\boldsymbol{S}^{j,\star}$ with $\boldsymbol{A}^j$ at the *j*-th iteration.
6: Solve the **ILP** problem to derive $\boldsymbol{A}^{j,\star}$ with the newly obtained $\boldsymbol{S}^{j,\star}$ at the *j*-th iteration.
7: Update $\boldsymbol{A}^j = \boldsymbol{A}^{j,\star}$, and $j = j+1$.
8: **until** Converge to the tolerated accuracy.
9: With the newly derived $\{\boldsymbol{A}_{i+1}, \boldsymbol{S}_{i+1}, \boldsymbol{Q}_{i+1}\}$, we optimize the communication bandwidth and computation resource allocation $\{\boldsymbol{B}_{i+1}, \boldsymbol{F}_{i+1}\}$ by solving $\mathbf{P_R1}$.
10: Update variables: $\boldsymbol{Q}_i = \boldsymbol{Q}_{i+1}$, $\boldsymbol{A}_i = \boldsymbol{A}_{i+1}$, $\boldsymbol{S}_i = \boldsymbol{S}_{i+1}$, $\boldsymbol{B}_i = \boldsymbol{B}_{i+1}$, $\boldsymbol{F}_i = \boldsymbol{F}_{i+1}$.
11: **until** Converge to the tolerated accuracy.

in the outer loop. The details of the overall algorithm are summarized in Algorithm 2, and its complexity analysis is given as follows. Denote I_1 and I_2 as the iteration number of the outer and inner loop, respectively. The computational complexity of $\mathbf{P_T}$, i.e., Algorithm 1, can be roughly given as $\mathcal{O}(I_1 K^3 N^3)$, where *K*, *N* are the number of the optimization variables in each iteration of Algorithm 1 [31], [44]. Since $\mathbf{P_S}$ consists of an ILP problem and an LP problem which can be solved by the interior point method in the polynomial time, while the inner loop computational complexity is $\mathcal{O}(2^{3KN})$ for the B&B algorithm [3]. Moreover, the complexity of $\mathbf{P_R}$ is $\mathcal{O}((2KN)^{3.5})$ due to the logarithmic form [45]. Therefore, the total computational complexity of the proposed overall algorithm is given as $\mathcal{O}(I_1(K^3N^3 + I_2 2^{3KN} + (2KN)^{3.5}))$.

V. Penalty Method-Based Low-Complexity Algorithm

Although **B&B** algorithm can find the globally optimal solution by traversing the entire variable space, which further makes the complexity of the algorithm unfavorable for practical applications with the increase of *K* and *N*, another algorithm is proposed to reduce computational complexity based on penalty method to deal with $\mathbf{P_S}$ in this section.

Firstly, the integer constraint (23n) in $\mathbf{P_S}$ can be equivalently transformed into

$$a_m^k[n] \in [0,1], \forall k, m, n, \tag{33a}$$

$$a_m^k[n](1 - a_m^k[n]) = 0, \forall k, m, n, \tag{33b}$$

which still do not change the non-convexity of $\mathbf{P_S}$ due to the strongly coupled variants. Thus, the auxiliary variable $\hat{\boldsymbol{A}} \triangleq \{\hat{\boldsymbol{A}}_1, \hat{\boldsymbol{A}}_2, \hat{\boldsymbol{A}}_3\}$, where $\hat{\boldsymbol{A}}_m \triangleq \{\hat{a}_m^k[n],\ \forall k, n\}$, is introduced to reformulate the coupling terms in both the objective function of $\mathbf{P_S}$ and constraints, i.e., (10), (14), (15) and (18), as

$$a_m^k[n] S_{\min} \le \hat{a}_m^k[n] \le a_m^k[n] S_{\max}, \forall k, m, n, \tag{34a}$$

$$s_k[n] - S_{\max}(1 - a_m^k[n]) \le \hat{a}_m^k[n] \le s_k[n], \forall k, m, n. \tag{34b}$$

To further settle $\mathbf{P_S}$, we define the following penalty function by penalizing the equality constraint (33b) into the objective function

$$f(\boldsymbol{A}, \hat{\boldsymbol{A}}, \boldsymbol{S}) = p_0(\hat{\boldsymbol{A}}, \boldsymbol{S}) + \mu p_1(\boldsymbol{A}), \tag{35}$$

where μ is the penalty parameter, $p_0(\hat{\boldsymbol{A}}, \boldsymbol{S})$ is the objective function of $\mathbf{P_S}$, and $p_1(\boldsymbol{A})$ represents the penalty term constituted by constraint (33b), given by

$$p_0(\hat{\boldsymbol{A}}, \boldsymbol{S}) \triangleq \sum_{k=1}^{K} \sum_{n=1}^{N} \hat{a}_2^k[n] \theta \eta f_k^u[n]^2 + \hat{a}_3^k[n] \omega, \tag{36a}$$

$$p_1(\boldsymbol{A}) \triangleq \sum_{k=1}^{K} \sum_{n=1}^{N} \sum_{m=1}^{3} g(a_m^k[n]), \tag{36b}$$

where

$$g(a_m^k[n]) \triangleq a_m^k[n](1 - a_m^k[n]), \forall k, m, n. \tag{37}$$

It is observed that $f(\boldsymbol{A}, \hat{\boldsymbol{A}}, \boldsymbol{S})$ is still non-convex due to the concavity of the penalized equality term, $g(a_m^k[n])$. Therefore, we approximate this term by utilizing the first-order Taylor expansion at the given point $a_{m,j}^k[n]$ at the *j*-th iteration,

$$\begin{aligned} g(a_m^k[n]) &\approx g(a_{m,j}^k[n]) + \nabla g(a_{m,j}^k[n])(a_m^k[n] - a_{m,j}^k[n]) \\ &= -a_{m,j}^k[n]^2 + (1 - 2a_{m,j}^k[n]) a_m^k[n]. \end{aligned} \tag{38}$$

Therefore, the objective function $f(\boldsymbol{A}, \hat{\boldsymbol{A}}, \boldsymbol{S})$ can be approximated at the *j*-th iteration by

$$\begin{aligned} \tilde{f}(\boldsymbol{A}, \hat{\boldsymbol{A}}, \boldsymbol{S}) = p_0(\hat{\boldsymbol{A}}, \boldsymbol{s}) + \mu \sum_{k=1}^{K} \sum_{n=1}^{N} \sum_{m=1}^{3} \\ \times \left\{ -a_{m,j}^k[n]^2 + \ (1 - 2a_{m,j}^k[n]) a_m^k[n] \right\}, \end{aligned} \tag{39}$$

and the sub-problem $\mathbf{P_S}$ can be rewritten as

$$\mathbf{P_S1}: \min_{\{\hat{\boldsymbol{A}}, \boldsymbol{A}, \boldsymbol{S}\}} \ \tilde{f}(\boldsymbol{A}, \hat{\boldsymbol{A}}, \boldsymbol{S}) \tag{40a}$$

$$\text{s.t.} \ \frac{\hat{a}_1^k[n]\theta_k}{f_k} + \frac{\hat{a}_2^k[n]\theta}{f_k^u[n]} + \frac{s_k[n]}{r_k[n]} - \frac{\hat{a}_1^k[n]}{r_k[n]} \le \tau, \forall k, n, \tag{40b}$$

$$\sum_{n=1}^{N} \left(\frac{\hat{a}_1^k[n]\theta_k}{f_k} + \frac{\hat{a}_2^k[n]\theta}{f_k^u[n]} + \frac{s_k[n]}{r_k[n]} - \frac{\hat{a}_1^k[n]}{r_k[n]} \right) \le T_k, \forall k, \tag{40c}$$

$$\sum_{n=1}^{N} \left(\frac{p_k\left(s_k[n] - \hat{a}_1^k[n]\right)}{r_k[n]} + \hat{a}_1^k[n]\theta_k \eta_k f_k^2 \right) \le E_k, \forall k, \tag{40d}$$

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

$$\sum_{n=1}^{N}\sum_{k=1}^{K}\hat{a}_3^k[n] \leq C, \tag{40e}$$

$$\sum_{n=1}^{N}\hat{a}_1^k[n] + \hat{a}_2^k[n] \geq S_k^{\text{exe}}, \forall k, \tag{40f}$$

$$0 \leq a_m^k[n] \leq 1, \forall k, m, n,$$
$$(23\text{a}),\ (23\text{b}),\ (23\text{j}),\ (23\text{m}),\ (34), \tag{40g}$$

which is LP with respect to $\{\hat{\boldsymbol{A}}, \boldsymbol{A}, \boldsymbol{S}\}$, and can be solved via well-developed toolboxs. Notice that this PM-based algorithm is tailored to reduce the computational complexity of the overall algorithm by replacing Step 4 to Step 8 in Algorithm 2. Moreover, the computational complexity of the PM-based algorithm is $\mathcal{O}(I_3\sqrt{7KN})$ [45], where I_3 denotes the number of iterations for the first-order Taylor expansion. Therefore, the complexity of the newly proposed overall algorithm can be reduced to $\mathcal{O}(I_1(K^3N^3 + I_3\sqrt{7KN} + (2KN)^{3.5}))$.

VI. Numerical Results

This section presents the numerical results to verify the effectiveness and performance of our proposed algorithm.

A. Parameter Settings

Several GNs are randomly distributed over a 200 m×200 m region, and each GN has a task waiting for further processing, the size of which is denoted by S_k and ranges from 50Kbits to 80Kbits. The flight time of the whole navigation between $\mathbf{Q}_0$ and $\mathbf{Q}_F$ is divided into the different number of time slots, N = 20/30/60/90, for comparison, and τ is the length of each slot, i.e., $\tau = T/N$ (in second). We set (θ, θ_k) as $(1, 1.2)\times 10^3$, and the CPU computation energy efficiency coefficients of UAV and GNs are given as $\eta = 10^{-26}$ and $\eta_k = 10^{-24}$, $\forall k$, referring to [46]–[48]. The simulation parameters are summarized in Table III. Three different offloading modes are considered in this section: 1) **Execution Offloading (EO) Mode**: all bits are computed at the UAV; 2) **Hybrid Task Scheduling (HTS) Mode**: the bits are either for computing or caching; and 3) **Caching Offloading (CO) Mode**: all bits are only for caching at the UAV.

TABLE III
Simulation Parameters

System Parameters	Values
Maximal CPU Frequency of UAV	4 GHz
CPU Frequency of GN	0.1 GHz
Total Bandwidth B	4 MHz
Caching Coefficient ω	1.5×10^{-8}
Caching capacity C	3×10^7 bits
Executed Bits S_k^{exe}	40% of total bits for GNs
Minimal Slot Bits $S_k^{\min}$	5×10^4 bits
Maximal Slot Bits $S_k^{\max}$	8×10^4 bits
Range of Bits at the k-th GN S_k	$[10, 72] \times 10^6$ bits
UAV Computing Coefficient η	10^{-26}
GN Computing Coefficient η_k	10^{-24}
Transmit Power of GN	0.1 W
Background Noise N_0	-160 dBm/Hz
Channel Gain at $d_0 = 1$ m	50 dB
UAV Related Parameters	**Values**
UAV Blade Power P_0 [26]	158.76 W
Induced Power P_i	88.63 W
Blade Speed Q_{tip}	120 m/s
Constant Parameter P_C	0.018
Maximal UAV speed $v_{\max}$	20 m/s
UAV Flight Height H	100 m

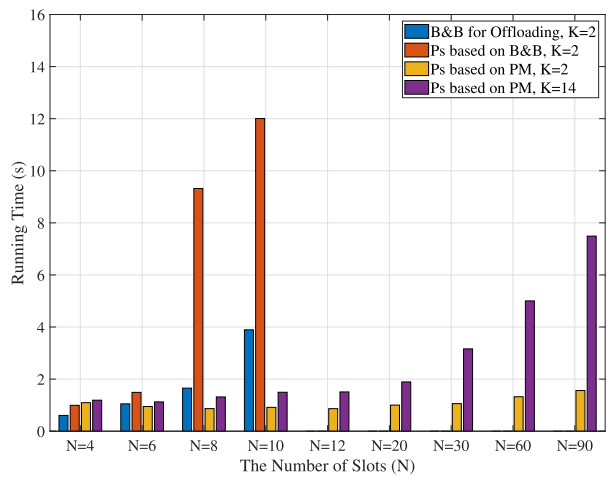


Fig. 1. The running time of B&B and PM for $\mathbf{P_S}$ with respect to the number of slots.

B. Results and Analysis

When the dimensions of GNs and slots are small, i.e., $K = 2$ and N = 4, 6, the running times of B&B and PM are nearly the same. PM can save more than 88% running time compared with B&B to solve the same problem when N = 8, 10 in Fig. 1.[3] Moreover, with the growing number of N, the running time of B&B massively increases due to the inner loop for solving ILP in $\mathbf{P_S}$ through the traversal search in the state space, which is intractable for practical applications with larger N. Especially, it usually takes several hours to find out the solution when $N = 12$, and may be infeasible sometimes. Fortunately, the total edge energy consumption (i.e., the energy consumption of execution for computing and caching at the UAV) yielded by PM is close to that of B&B in Fig. 2, where the UAV trajectory is fixed and all bits are computed at GNs or the UAV. In addition, the total edge energy consumption obtained by "$\mathbf{P_S}$ based on PM" is much less than that derived by *Equal Resource Allocation*, i.e., the computation resource at the UAV is equivalently assigned to GNs. Since the B&B algorithm is more time-consuming and makes the complexity unfavorable with the increasing K and N, *PM has been employed to deal with subproblem* $\mathbf{P_S}$ *in the simulations*, which has relatively satisfactory performance compared with B&B.

The UAV trajectories with different offloading modes are shown in Fig. 3, which are nearly the same. Since UAV can obtain better channel gain and reduce the communication delay via approaching GNs to complete the task within the given latency, the trajectory of **EO** is closer to GNs than the other two modes. It is observed that *different offloading modes have*

[3]Notice that PM can solve task decision and bits scheduling simultaneously in $\mathbf{P_S}$, and B&B is used to solve task decision. In Fig. 1, 'B&B for offloading' means that we ONLY solve ILP (i.e., task decision) via B&B in $\mathbf{P_S}$, meanwhile, we iteratively optimize LP and ILP in '$\mathbf{P_S}$ based on B&B'.

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

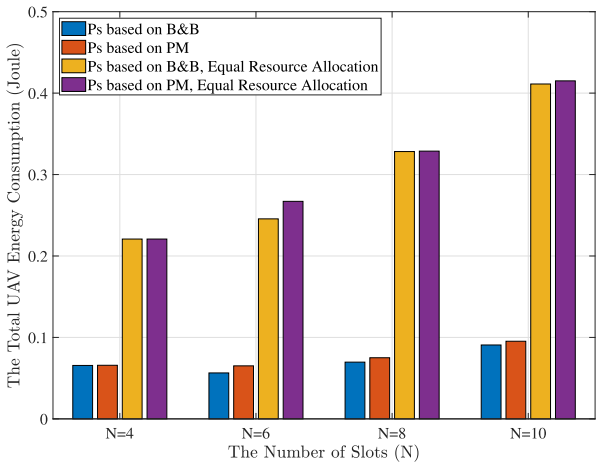


Fig. 2. The total edge energy consumption with respect to B&B and PM for P_s where $K = 2$, $\tau = 1$ s.

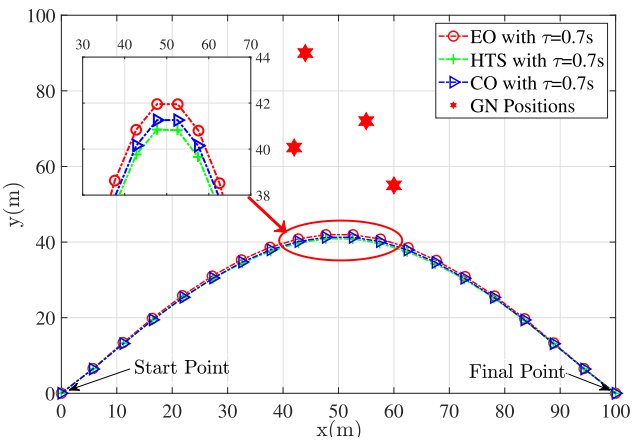


Fig. 3. The trajectories of UAV with respect to different offloading modes, where $K = 4$, $N = 20$.

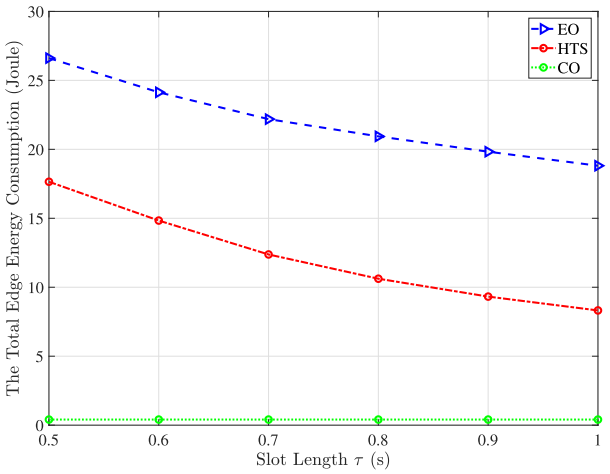


Fig. 4. The total edge energy consumption of different offloading modes with respect to the latency requirement, where $K = 14$, $N = 30$.

less influence on the trajectory, but have a great effect on the total edge energy consumption shown in Fig. 4, Fig. 5, and Fig. 6.

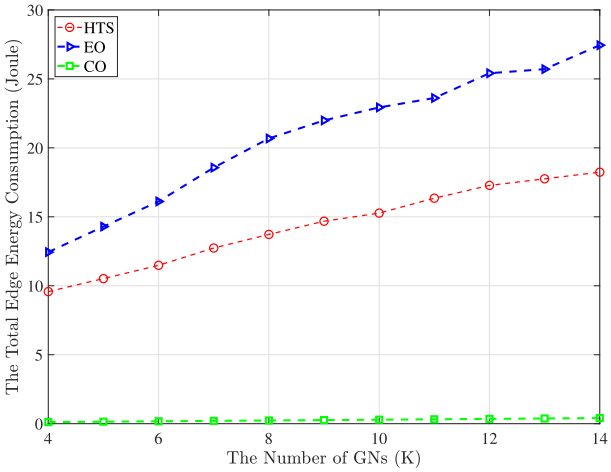


Fig. 5. The total edge energy consumption of different offloading modes with respect to the number of GNs, where $N = 30$, $\tau = 0.5$ s.

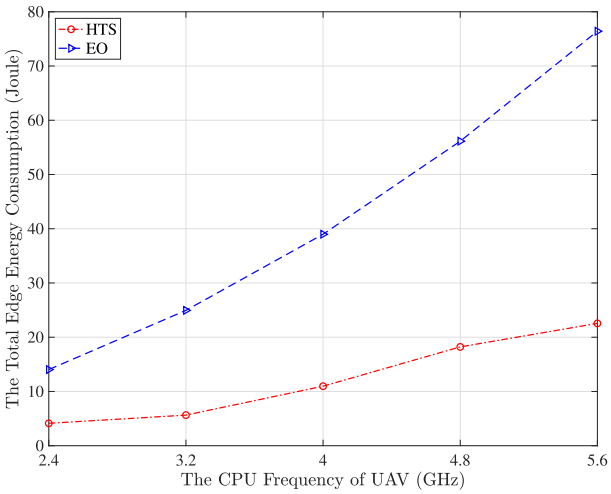


Fig. 6. The total edge energy consumption of different offloading modes with respect to the CPU frequency of UAV, where $K = 8$, $N = 30$, $\tau = 0.5$ s, $f_k = 0.2$ GHz, $\forall k$.

In Fig. 4, we present the total edge energy consumption of different offloading mechanisms for the various latency conditions, where all GNs' tasks are offloaded for remote processing. With more strict latency requirements, the energy consumption of execution is increasing, because more computing resources are needed to satisfy the latency requirements. Since caching consumes less energy than computing operation to process the same number of bits, the majority of energy is consumed by **EO**, and a small portion of the energy is consumed by **CO**, while **HTS** falls in between. Besides, the UAV can choose to cache more bits when it is under low energy conditions, and will compute them in the future. Due to the negligible time for cache writing, the edge energy consumption in **CO** mode is invariable concerning different given latency requirements since all bits are cached. With the growing number of GNs, the total edge energy consumption is increasing since more bits are offloaded to the UAV for computation or caching observed from Fig. 5. Moreover, with the growing CPU frequency of UAV, the UAV has more computing power

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

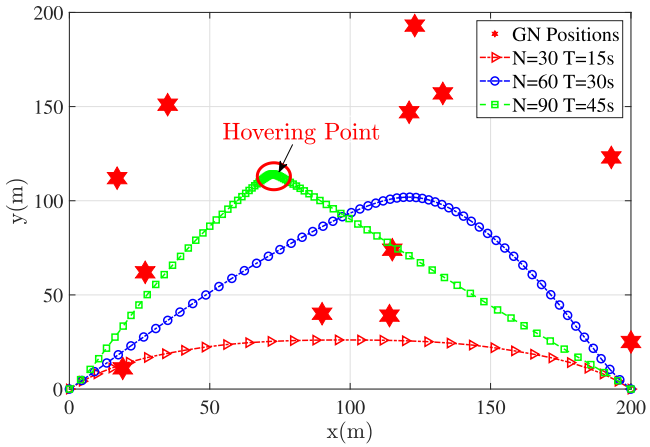


Fig. 7. The trajectories of UAV with respect to different numbers of slots where $K = 12$, $\tau = 0.5$ s.

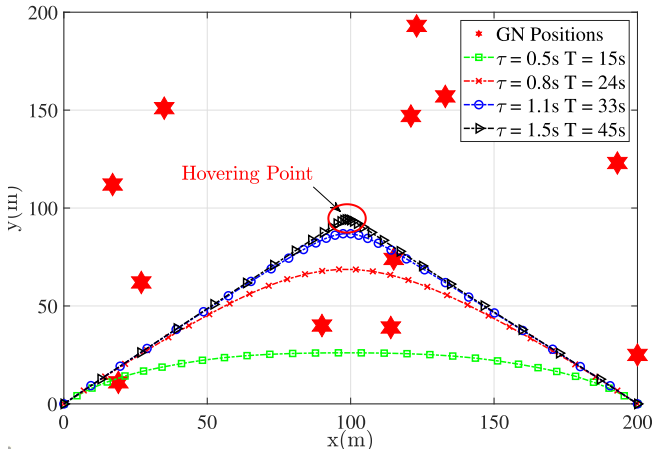


Fig. 8. The trajectories of UAV with respect to different slot lengths where $K = 12$, $N = 30$.

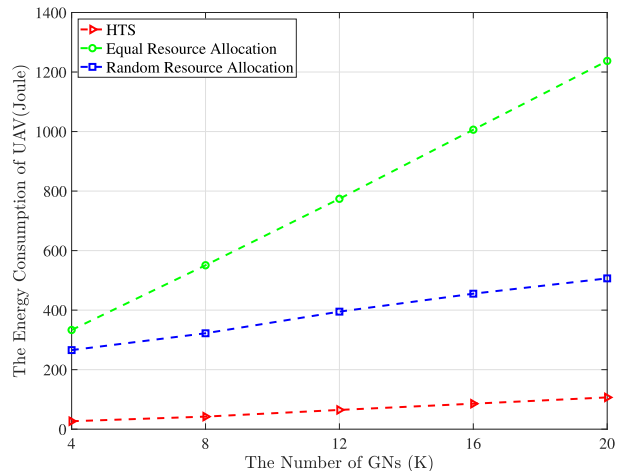


Fig. 9. The comparisons of the energy consumption at the UAV versus different numbers of GNs where $N = 20$, $\tau = 0.5$ s for different algorithms.

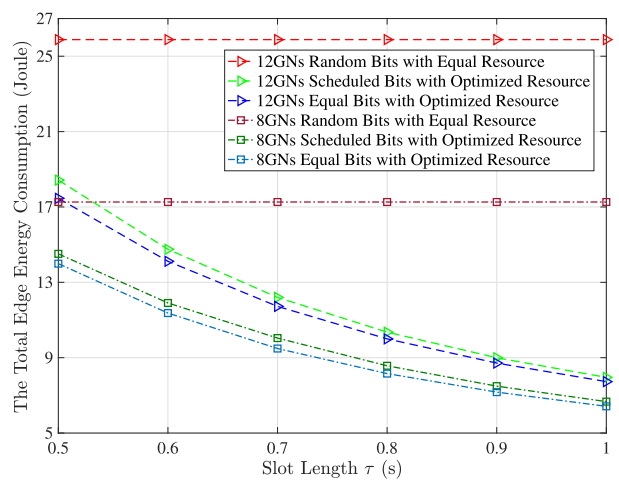


Fig. 10. The total edge energy consumption of different algorithms with respect to different latency requirements, where $N = 30$.

to process the data from GNs, and thus consumes more energy for executing in Fig. 6. It is observed that **HTS** mode can save 67.5%–77.4% energy consumption compared with **EO** mode, because caching operation consumes less energy according to (20). Since **HTS** mode is more general, i.e., the combination of **EO** and **CO** mode, **HTS** mode has been adopted in the following simulations, unless otherwise mentioned.

Fig. 7 and Fig. 8 present the UAV trajectories with respect to different numbers of slots and different slot lengths, respectively. With the increasing N, the UAV has sufficient time to serve GNs, and thus it can fly closer to GNs to further reduce the uplink transmission delay and provide better services. Especially, it even can hover to guarantee a more stable uploading channel in Fig. 7. Moreover, with the gradually decreasing latency requirement, i.e., τ is increasing shown in Fig. 8, because GNs can have more time to deal with their tasks, instead of offloading to the UAV, which can help save computation energy at the UAV. Meanwhile, the UAV can have extra energy to approach GNs and obtain better channel gains for helping GNs.

In Fig. 9, for the proposed and reference algorithms, we plot the energy consumption of UAV according to different numbers of GNs. With the increasing number of users, the energy consumption of UAV for different algorithms increases, and HTS can help save 17.7% – 54.6% and 78.9% – 91.9% energy consumption compared with *Equal Resource Allocation*, i.e., the computation resource at the UAV is equivalently distributed to GNs, and *Random Resource Allocation*, i.e., the computation resource at the UAV is randomly distributed to GNs. Notice that the two methods employ the same techniques proposed in this paper to obtain $\boldsymbol{Q}$, $\boldsymbol{A}$, $\boldsymbol{S}$ and $\boldsymbol{B}$, except $\boldsymbol{F}$. Since the computation resource assigned to GNs is more than enough, computation resource misallocation will result in more energy consumption for executing tasks, which increases quadratically with respect to computation frequency observed from (19). Moreover, with the growing number of GNs, the unreasonable computation resource allocation will cause more energy waste, especially for Equal Resource Allocation.

In Fig. 10, we compare the total edge energy consumption of different algorithms concerning different latency requirements in EO mode, where three different (i.e., random, equal, and scheduled) strategies are investigated. To be specific, task bits are randomly distributed over different slots and computed with equal computation resource for the random one, each GN

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

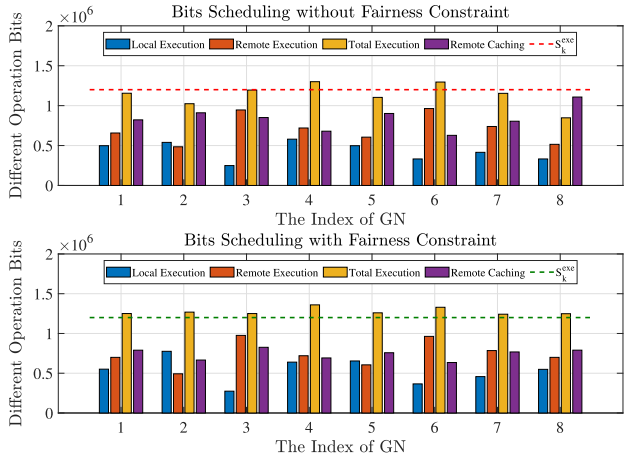


Fig. 11. The bits for different operations with or without fairness constraint where $K = 8$, $N = 20$, $\tau = 0.5$ s.

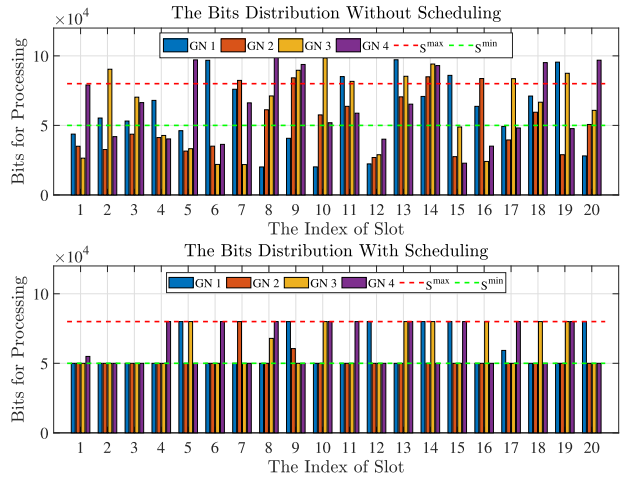


Fig. 12. The distribution of bits in each slot before and after bits scheduling where $K = 4$, $N = 20$, $\tau = 0.5$ s.

has the same number of bits on all slots for the equal one, and the bits distribution is optimized according to PM for the scheduled one, where bits are processed with optimized computation resource in the last two strategies. As shown in Fig. 10, the increasing number of GNs will bring more bits to be processed by the UAV, and thus consumes more energy. The scheduled strategy can arrange a reasonable number of bits in each slot and assign a moderate amount of computation resource for GNs according to the physical situations (i.e., channel gains, bandwidth, and trajectory), which help save energy. Therefore, our proposed algorithm outperforms the others with less energy consumption, saving 17%−25% energy, since computing resource will be wasted when the assigned bits are not enough, and insufficient if more bits are allocated in the equal case. Moreover, the energy consumption is decreasing with the decreasing latency requirement of GNs.

Since fairness is one design concern in this paper, the demand of the number of processed bits for computing operation should be satisfied according to fairness constraint (9). As indicated by Fig. 11, we compare the bits for different operations with or without fairness constraints. It can be seen that the number of bits for different operations is relatively random without fairness constraint, and the number of the total execution bits does not reach the target S_k^{exe}. However, when fairness constraint (9) is active, the number of executed bits approaches the target value, which can guarantee fairness among different operations of tasks.

In Fig. 12, we compare the bits distribution for different operations before and after scheduling for different slots in EO mode. It has been observed that the bits distribution before scheduling is stochastic, which is against the fairness constraint (6). To be specific, the scheduled bits of some GNs in most cases are less than $S_k^{\min}$, and more than $S_k^{\max}$ in a few cases, which cannot guarantee the fairness among GNs, and the UAV may be overwhelmed by the numerous data from these GNs. Fortunately, it becomes more stable after scheduling, i.e., the distribution of bits is relatively concentrated during the UAV navigation when the UAV approaches GNs. According to Fig. 3, when the UAV is far away from GNs, GNs will offload fewer bits for computing due to the poor channel gains, but schedule more bits to upload with better channel conditions. That is, task scheduling enables GNs to transmit more bits with better channel conditions to ask the UAV for an assistant. With the help of bit scheduling, GNs will make full advantage of computation and caching resources at the UAV to get better services, and the UAV-enabled MEC network will become more stable without bursting since the number of scheduled bits is more balanced.

VII. Conclusion

In this paper, we formulated the UAV's energy consumption minimization problem as an MINLP problem, where the UAV's trajectory, resource allocation, task decision, and bits scheduling are jointly considered with the consideration of fairness. To tackle this complex problem, an iterative framework, which leverages SCA and BCD methods, is adopted to find the sub-optimal solution to our optimization problem. Due to the high complexity of solving the task decision and bits scheduling subproblem, a penalty method-based low-complexity algorithm is proposed to solve this subproblem instead of B&B algorithm, which can save more than 88% running time and achieve relatively satisfactory performance compared with B&B. Simulation results show that our proposed algorithm could efficiently reduce the energy consumption of UAV, and help save 17.7% – 54.6% and 78.9% – 91.9% energy consumption compared with the other benchmarks.

Although our iterative algorithm can obtain satisfactory performance in terms of reducing the energy consumption of UAV, the employment of beamforming technique may further save energy according to the design of transmitter and receiver [49], [50], improve the energy efficiency [49], and reduce the end-to-end latency [50]. Thus, we will apply the related techniques in our future works.

References

[1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 4th Quart., 2017.

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

[2] J. Zhang *et al.*, "Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2633–2645, Aug. 2018.

[3] J. Zhang *et al.*, "Joint resource allocation for latency-sensitive services over mobile edge computing networks with caching," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4283–4294, Jun. 2019.

[4] W. Wen, Y. Cui, T. Q. S. Quek, F.-C. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.

[5] M. Zhao *et al.*, "Energy-aware offloading in time-sensitive networks with mobile edge computing," 2020. [Online]. Available: https://arxiv.org/abs/2003.12719.

[6] B. Li, Z. Fei, and Y. Zhang, "UAV communications for 5G and beyond: Recent advances and future trends," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2241–2263, Apr. 2019.

[7] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.

[8] F. Cheng *et al.*, "UAV trajectory optimization for data offloading at the edge of multiple cells," *IEEE Trans. Veh. Technol.*, vol. 67, no. 7, pp. 6732–6736, Jul. 2018.

[9] X. Diao, J. Zheng, Y. Cai, Y. Wu, and A. Anpalagan, "Fair data allocation and trajectory optimization for UAV-assisted mobile edge computing," *IEEE Commun. Lett.*, vol. 23, no. 12, pp. 2357–2361, Dec. 2019.

[10] J. Hu, M. Jiang, Q. Zhang, Q. Li, and J. Qin, "Joint optimization of UAV position, time slot allocation, and computation task partition in multiuser aerial mobile-edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 7, pp. 7231–7235, Jul. 2019.

[11] L. Zhang and N. Ansari, "Latency-aware IoT service provisioning in UAV-aided mobile-edge computing networks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10573–10580, Oct. 2020.

[12] Q. Hu, Y. Cai, G. Yu, Z. Qin, M. Zhao, and G. Y. Li, "Joint offloading and trajectory design for UAV-enabled mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1879–1892, Apr. 2019.

[13] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7808–7822, Aug. 2020.

[14] J. Wang, K. Liu, and J. Pan, "Online UAV-mounted edge server dispatching for mobile-to-mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1375–1386, Feb. 2020.

[15] M. Hua, Y. Wang, C. Li, Y. Huang, and L. Yang, "UAV-aided mobile edge computing systems with one by one access scheme," *IEEE Trans. Green Commun. Netw.*, vol. 3, no. 3, pp. 664–678, Sep. 2019.

[16] R. A. C. da Silva and N. L. S. da Fonseca, "Location of fog nodes for reduction of energy consumption of end-user devices," *IEEE Trans. Green Commun. Netw.*, vol. 4, no. 2, pp. 593–605, Jun. 2020.

[17] W.-T. Li *et al.*, "Collaborative offloading for UAV-enabled time-sensitive MEC networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2021, no. 1, pp. 1–17, 2021.

[18] Y. Du, K. Wang, K. Yang, and G. Zhang, "Energy-efficient resource allocation in UAV Based MEC system for IoT devices," in *Proc. IEEE Global Commun. Conf.*, Abu Dhabi, UAE, 2018, pp. 1–6.

[19] M. Hua, Y. Huang, Y. Wang, Q. Wu, H. Dai, and L. Yang, "Energy optimization for cellular-connected multi-UAV mobile edge computing systems with multi-access schemes," *J. Commun. Inf. Netw.*, vol. 3, no. 4, pp. 33–44, Dec. 2018.

[20] Y. Du, K. Yang, K. Wang, G. Zhang, Y. Zhao, and D. Chen, "Joint resources and workflow scheduling in UAV-enabled Wirelessly-powered MEC for IoT systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10187–10200, Oct. 2019.

[21] H. Mei, K. Yang, Q. Liu, and K. Wang, "Joint trajectory-resource optimization in UAV-enabled edge-cloud system with virtualized mobile clone," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5906–5921, Jul. 2020.

[22] Y. Liu, K. Xiong, Q. Ni, P. Fan, and K. B. Letaief, "UAV-assisted wireless powered cooperative mobile edge computing: Joint offloading, CPU control, and trajectory optimization," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2777–2790, Apr. 2020.

[23] J. Xiong, H. Guo, and J. Liu, "Task offloading in UAV-aided edge computing: Bit allocation and trajectory optimization," *IEEE Commun. Lett.*, vol. 23, no. 3, pp. 538–541, Mar. 2019.

[24] X. Hu, K.-K. Wong, K. Yang, and Z. Zheng, "UAV-assisted relaying and edge computing: Scheduling and trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 18, no. 10, pp. 4738–4752, Oct. 2019.

[25] J. Zhang *et al.*, "Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3688–3699, Apr. 2019.

[26] T. Zhang, Y. Xu, J. Loo, D. Yang, and L. Xiao, "Joint computation and communication design for UAV-assisted mobile edge computing in IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5505–5516, Aug. 2020.

[27] X. Zhang, J. Zhang, J. Xiong, L. Zhou, and J. Wei, "Energy-efficient multi-UAV-enabled multiaccess edge computing incorporating NOMA," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5613–5627, Jun. 2020.

[28] Y. K. Tun, Y. M. Park, N. H. Tran, W. Saad, S. R. Pandey, and C. S. Hong, "Energy-efficient resource management in UAV-assisted mobile edge computing," *IEEE Commun. Lett.*, vol. 25, no. 1, pp. 249–253, Jan. 2021.

[29] H. Guo and J. Liu, "UAV-enhanced intelligent offloading for Internet of Things at the edge," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2737–2746, Apr. 2020.

[30] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.

[31] C. Zhan and Y. Zeng, "Aerial–ground cost tradeoff for multi-UAV-enabled data collection in wireless sensor networks," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1937–1950, Mar. 2020.

[32] J. Zhang *et al.*, "Computation-efficient offloading and trajectory scheduling for multi-UAV assisted mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2114–2125, Feb. 2020.

[33] C. Wang, C. Liang, F. R. Yu, Q. Chen, and L. Tang, "Computation offloading and resource allocation in wireless cellular networks with mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 16, no. 8, pp. 4924–4938, Aug. 2017.

[34] J. Ji, K. Zhu, D. Niyato, and R. Wang, "Joint cache placement, flight trajectory, and transmission power optimization for multi-UAV assisted wireless networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5389–5403, Aug. 2020.

[35] J. Ji, K. Zhu, D. Niyato, and R. Wang, "Joint trajectory design and resource allocation for secure transmission in cache-enabled UAV-relaying networks with D2D communications," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1557–1571, Feb. 2021.

[36] Y. Wang, S. Min, X. Wang, W. Liang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016.

[37] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018.

[38] J.-J. Yu, M. Zhao, W.-T. Li, D. Liu, S. Yao, and W. Feng, "Joint offloading and resource allocation for time-sensitive multi-access edge computing network," in *Proc. IEEE Wireless Commun. Netw. Conf.*, Seoul, South Korea, 2020, pp. 1–6.

[39] F. Gabry, V. Bioglio, and I. Land, "On energy-efficient edge caching in heterogeneous networks," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3288–3298, Dec. 2016.

[40] N. Choi, K. Guan, D. C. Kilper, and G. Atkinson, "In-network caching effect on optimal energy consumption in content-centric networking," in *Proc. IEEE Int. Conf. Commun.*, Ottawa, ON, Canada, 2012, pp. 2889–2894.

[41] P. Richtárik and M. Takáč, "Iteration complexity of randomized block-coordinate descent methods for minimizing a composite function," *Math. Program.*, vol. 144, nos. 1–2, pp. 1–38, 2014.

[42] M. Zhao, J. Y. Ryu, J. Lee, T. Q. S. Quek, and S. Feng, "Exploiting trust degree for multiple-antenna user cooperation," *IEEE Trans. Wireless Commun.*, vol. 16, no. 8, pp. 4908–4923, Aug. 2017.

[43] M. Grant and S. Boyd, *CVX: MATLAB Software for Disciplined Convex Programming, Version 2.1*, CVX Res., Inc., Austin, TX, USA, 2014.

[44] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[45] M. Hua, L. Yang, Q. Wu, and A. L. Swindlehurst, "3D UAV trajectory and communication design for simultaneous uplink and downlink transmission," *IEEE Trans. Commun.*, vol. 68, no. 9, pp. 5908–5923, Sep. 2020.

[46] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Sep. 2013.

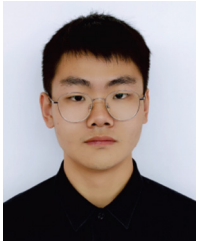
[47] K. Kumar and Y.-H. Lu, "Cloud computing for mobile users: Can offloading computation save energy?" *Computer*, vol. 43, no. 4, pp. 51–56, Apr. 2010.

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.

[48] Y. Wu, Y. Wang, F. Zhou, and R. Q. Hu, "Computation efficiency maximization in OFDMA-based mobile edge computing networks," *IEEE Commun. Lett.*, vol. 24, no. 1, pp. 159–163, Jan. 2020.
[49] A. Kaushik, E. Vlachos, C. Tsinos, J. Thompson, and S. Chatzinotas, "Joint bit allocation and hybrid beamforming optimization for energy efficient millimeter wave MIMO systems," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 1, pp. 119–132, Mar. 2021.
[50] K. Guo and T. Q. S. Quek, "On the asynchrony of computation offloading in multi-user MEC systems," *IEEE Trans. Commun.*, vol. 68, no. 12, pp. 7746–7761, Dec. 2020.

Mingxiong Zhao (Member, IEEE) received the B.S. degree in electrical engineering and the Ph.D. degree in information and communication engineering from the South China University of Technology, Guangzhou, China, in 2011 and 2016, respectively. He was a visiting Ph.D. student with the University of Minnesota, Twin Cities, MN, USA, from 2012 to 2013 and the Singapore University of Technology and Design, Singapore, from 2015 to 2016, respectively. Since 2016, he has been with the School of Software, Yunnan University, Kunming, China, where he is currently an Associate Professor. His current research interests include physical layer security, mobile edge computing, and edge AI techniques.

Wentao Li received the B.S. degree in software engineering and the M.S. degree in artificial intelligent and machine learning from Yunnan University, Kunming, China, in 2014 and 2021, respectively. His research interest are UAV-enable computing and convex optimization.



Lingyan Bao was born on 1998 in Qujing, China. He received the B.S. degree in information security from Yunnan University, Kunming, China, in 2020, where he is currently pursuing the M.S. degree in electronic information. His research interest are UAV-enabled communications, resource allocation, and edge AI techniques.

Jia Luo received the B.S. degree in information security from Yunnan University, Kunming, China, in 2020, where she is currently pursuing the M.S. degree in cyberspace security. Her research interest is UAV-enabled communications and federated learning.

Zhenli He received the Ph.D. degree in systems analysis and integration from Yunnan University, Kunming, China, in 2015, where he is currently a Lecturer. His current research interests include edge computing, energy-efficient computing, heterogeneous computing, and machine learning.

Di Liu (Member, IEEE) received the B.Eng. and M.Eng. degrees from Northwestern Polytechnical University, Xi'an, China, in 2007 and 2011, respectively, and the Ph.D. degree from Leiden University, Leiden, The Netherlands, in 2017. He is currently an Assistant Professor with the School of Software, Yunnan University, China, and also a Research Fellow with HP-NTU Corp Lab, Nanyang Technological University, Singapore. His research interests include the fields of edge systems, machine learning on embedded systems, energy-efficient multicore/many core systems, and cyber-physical systems.

Authorized licensed use limited to: SRM University Amaravathi. Downloaded on April 15,2026 at 05:37:47 UTC from IEEE Xplore. Restrictions apply.